

LED Control (LEDC)

[\[中文\]](#)

Introduction

The LED control (LEDC) peripheral is primarily designed to control the intensity of LEDs, although it can also be used to generate PWM signals for other purposes. It has 8 channels which can generate independent waveforms that can be used, for example, to drive RGB LED devices.

LEDC channels are divided into two groups of 8 channels each. One group of LEDC channels operates in high speed mode. This mode is implemented in hardware and offers automatic and glitch-free changing of the PWM duty cycle. The other group of channels operate in low speed mode, the PWM duty cycle must be changed by the driver in software. Each group of channels is also able to use different clock sources.

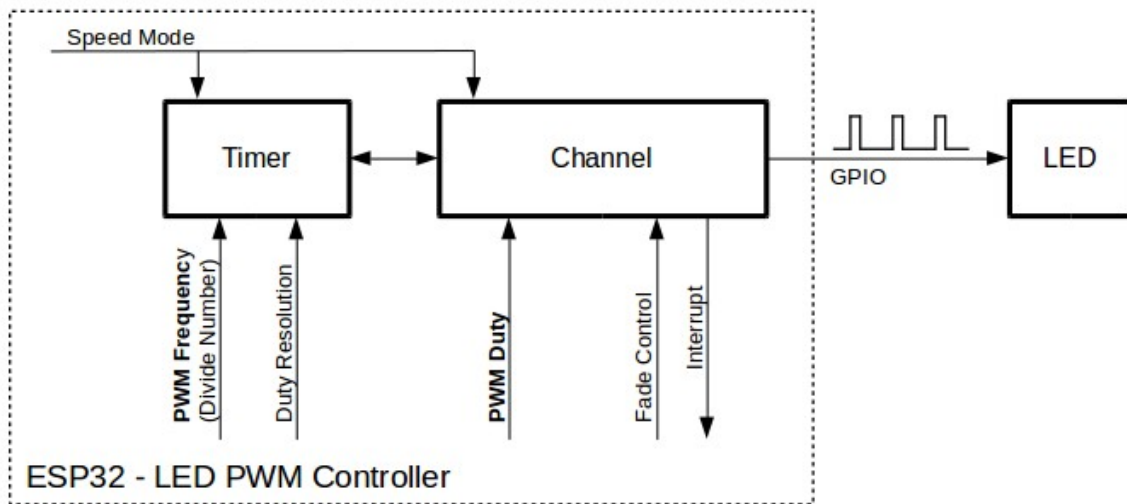
The PWM controller can automatically increase or decrease the duty cycle gradually, allowing for fades without any processor interference.

Functionality Overview

Setting up a channel of the LEDC in either [high or low speed mode](#) is done in three steps:

1. [Timer Configuration](#) by specifying the PWM signal's frequency and duty cycle resolution.
2. [Channel Configuration](#) by associating it with the timer and GPIO to output the PWM signal.
3. [Change PWM Signal](#) that drives the output in order to change LED's intensity. This can be done under the full control of software or with hardware fading functions.

As an optional step, it is also possible to set up an interrupt on fade end.



Key Settings of LED PWM Controller's API

Note

For an initial setup, it is recommended to configure for the timers first (by calling `ledc_timer_config()`), and then for the channels (by calling `ledc_channel_config()`). This ensures the PWM frequency is at the desired value since the appearance of the PWM signal from the IO pad.

Timer Configuration

Setting the timer is done by calling the function `ledc_timer_config()` and passing the data structure `ledc_timer_config_t` that contains the following configuration settings:

- Speed mode `ledc_mode_t`
- Timer number `ledc_timer_t`
- PWM signal frequency in Hz
- Resolution of PWM duty
- Source clock `ledc_clk_cfg_t`

The frequency and the duty resolution are interdependent. The higher the PWM frequency, the lower the duty resolution which is available, and vice versa. This relationship might be important if you are planning to use this API for purposes other than changing the intensity of LEDs. For more details, see Section [Supported Range of Frequency and Duty Resolutions](#).

The source clock can also limit the PWM frequency. The higher the source clock frequency, the higher the maximum PWM frequency can be configured.

Characteristics of ESP32 LEDC source clocks

Clock name	Clock freq	Speed mode	Clock capabilities
APB_CLK	80 MHz	High / Low	/
REF_TICK	1 MHz	High / Low	Dynamic Frequency Scaling compatible
RC_FAST_CLK	~ 8 MHz	Low	Dynamic Frequency Scaling compatible, Light-sle

Note

1. On ESP32, if RC_FAST_CLK is chosen as the LEDC clock source, an internal calibration will be performed to get the exact frequency of the clock. This ensures the accuracy of output PWM signal frequency.

The LEDC driver offers a helper function `ledc_find_suitable_duty_resolution()` to find the maximum possible resolution for the timer, given the source clock frequency and the desired PWM signal frequency.

When a timer is no longer needed by any channel, it can be deconfigured by calling the same function `ledc_timer_config()`. The configuration structure `ledc_timer_config_t` passes in should be:

- `ledc_timer_config_t::speed_mode` The speed mode of the timer which wants to be deconfigured belongs to (`ledc_mode_t`)
- `ledc_timer_config_t::timer_num` The ID of the timers which wants to be deconfigured (`ledc_timer_t`)
- `ledc_timer_config_t::deconfigure` Set this to true so that the timer specified can be deconfigured

Channel Configuration

When the timer is set up, configure the desired channel (one out of `ledc_channel_t`). This is done by calling the function `ledc_channel_config()`.

Similar to the timer configuration, the channel setup function should be passed a structure `ledc_channel_config_t` that contains the channel's configuration parameters.

At this point, the channel should start operating and generating the PWM signal on the selected GPIO, as configured in `ledc_channel_config_t`, with the frequency specified in the timer settings and the given duty cycle. The channel operation (signal generation) can be suspended at any time by calling the function `ledc_stop()`.

Change PWM Signal

Once the channel starts operating and generating the PWM signal with the constant duty cycle and frequency, there are a couple of ways to change this signal. When driving LEDs, primarily the duty cycle is changed to vary the light intensity.

The following two sections describe how to change the duty cycle using software and hardware fading. If required, the signal's frequency can also be changed; it is covered in Section [Change PWM Frequency](#).

Change PWM Duty Cycle Using Software

To set the duty cycle, use the dedicated function `ledc_set_duty()`. After that, call `ledc_update_duty()` to activate the changes. To check the currently set value, use the corresponding `_get_` function `ledc_get_duty()`.

Another way to set the duty cycle, as well as some other channel parameters, is by calling `ledc_channel_config()` covered in Section [Channel Configuration](#).

The range of the duty cycle values passed to functions depends on selected `duty_resolution` and should be from `0` to `(2 ** duty_resolution)`. For example, if the selected duty resolution is 10, then the duty cycle values can range from 0 to 1024. This provides the resolution of ~ 0.1%.

⚠ Warning

On ESP32, when channel's binded timer selects its maximum duty resolution, the duty cycle value cannot be set to `(2 ** duty_resolution)`. Otherwise, the internal duty counter in the hardware will overflow and be messed up.

Change PWM Duty Cycle Using Hardware

The LEDC hardware provides the means to gradually transition from one duty cycle value to another. To use this functionality, enable fading with `ledc_fade_func_install()` and then configure it by calling one of the available fading functions:

- `ledc_set_fade_with_time()`
- `ledc_set_fade_with_step()`
- `ledc_set_fade()`

Start fading with `ledc_fade_start()`. A fade can be operated in blocking or non-blocking mode, please check `ledc_fade_mode_t` for the difference between the two available fade modes. Note that with either fade mode, the next fade or fixed-duty update will not take effect until the last fade finishes. Due to hardware limitations, there is no way to stop a fade before it reaches its target duty.

To get a notification about the completion of a fade operation, a fade end callback function can be registered for each channel by calling `ledc_cb_register()` after the fade service being installed. The fade end callback prototype is defined in `ledc_cb_t`, where you should return a boolean value from the callback function, indicating whether a high priority task is woken up by this callback function. It is worth mentioning, the callback and the function invoked by itself should be placed in IRAM, as the interrupt service routine is in IRAM.

`ledc_cb_register()` will print a warning message if it finds the addresses of callback and user context are incorrect.

If not required anymore, fading and an associated interrupt can be disabled with

```
ledc_fade_func_uninstall().
```

Change PWM Frequency

The LEDC API provides several ways to change the PWM frequency "on the fly":

- Set the frequency by calling `ledc_set_freq()`. There is a corresponding function `ledc_get_freq()` to check the current frequency.
- Change the frequency and the duty resolution by calling `ledc_bind_channel_timer()` to bind some other timer to the channel.
- Change the channel's timer by calling `ledc_channel_config()`.

More Control Over PWM

There are several individual timer-specific functions that can be used to change PWM output:

- `ledc_timer_rst()`
- `ledc_timer_pause()`
- `ledc_timer_resume()`

The first function is called "behind the scenes" by `ledc_timer_config()` to provide a startup of a timer after it is configured.

Power Management

LEDC driver does not utilize power management lock to prevent the system from going into Light-sleep. Instead, the LEDC peripheral power domain state and the PWM signal output behavior during sleep can be chosen by configuring `ledc_channel_config_t::sleep_mode`. The default mode is `LEDC_SLEEP_MODE_NO_ALIVE_NO_PD`, which stands for no signal output and LEDC power domain will not be powered down during sleep.

If signal output needs to be maintained in Light-sleep, then select

`LEDC_SLEEP_MODE_KEEP_ALIVE`. As long as the binded LEDC timer clock source is Light-sleep compatible, the PWM signal can continue its output even the system enters Light-sleep. The cost is a higher power consumption in sleep, since the clock source and the power domain where LEDC belongs to cannot be powered down. Note that, if there is an unfinished fade before entering sleep, the fade can also continue during sleep, but the target duty might not be reached exactly. It will adjust to the target duty after wake-up.

LEDC High and Low Speed Mode

High speed mode enables a glitch-free changeover of timer settings. This means that if the timer settings are modified, the changes will be applied automatically on the next overflow interrupt of the timer. In contrast, when updating the low-speed timer, the change of settings should be explicitly triggered by software. The LEDC driver handles it in the background, e.g., when `ledc_timer_config()` is called.

For additional details regarding speed modes, see [ESP32 Technical Reference Manual > LED PWM Controller \(LEDC\)](#) [PDF].

Supported Range of Frequency and Duty Resolutions

The LED PWM Controller is designed primarily to drive LEDs. It provides a large flexibility of PWM duty cycle settings. For instance, the PWM frequency of 5 kHz can have the maximum duty resolution of 13 bits. This means that the duty can be set anywhere from 0 to 100% with a resolution of $\sim 0.012\%$ ($2^{13} = 8192$ discrete levels of the LED intensity). Note, however, that these parameters depend on the clock signal clocking the LED PWM Controller timer which in turn clocks the channel (see [timer configuration](#) and the [ESP32 Technical Reference Manual > LED PWM Controller \(LEDC\)](#) [PDF]).

The LEDC can be used for generating signals at much higher frequencies that are sufficient enough to clock other devices, e.g., a digital camera module. In this case, the maximum available frequency is 40 MHz with duty resolution of 1 bit. This means that the duty cycle is fixed at 50% and cannot be adjusted.

The LEDC API is designed to report an error when trying to set a frequency and a duty resolution that exceed the range of LEDC's hardware. For example, an attempt to set the frequency to 20 MHz and the duty resolution to 3 bits results in the following error reported on a serial monitor:

```
E (196) ledc: requested frequency and duty resolution cannot be achieved, try reducing
freq_hz or duty_resolution. div_param=128
```

In such a situation, either the duty resolution or the frequency must be reduced. For example, setting the duty resolution to 2 resolves this issue and makes it possible to set the duty cycle

The LEDC driver also captures and reports attempts to configure frequency/duty resolution combinations that are below the supported minimum, e.g.,:

```
E (196) ledc: requested frequency and duty resolution cannot be achieved, try
increasing freq_hz or duty_resolution. div_param=128000000
```

The duty resolution is normally set using `ledc_timer_bit_t`. This enumeration covers the range from 10 to 15 bits. If a smaller duty resolution is required (from 10 down to 1), enter the equivalent numeric values directly.

Application Example

- [peripherals/ledc/ledc_basic](#) demonstrates how to use the LEDC to generate a PWM signal in LOW SPEED mode.
- [peripherals/ledc/ledc_fade](#) demonstrates how to control the intensity of LEDs using the LEDC fade functionality.

API Reference

Header File

- [components/esp_driver_ledc/include/driver/ledc.h](#)
- This header file can be included with:

```
#include "driver/ledc.h"
```

- This header file is a part of the API provided by the `esp_driver_ledc` component. To declare that your component depends on `esp_driver_ledc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_ledc
```

or

```
PRIV_REQUIRES esp_driver_ledc
```

Functions

`esp_err_t ledc_channel_config(const ledc_channel_config_t *ledc_conf)`

LEDC channel configuration Configure LEDC channel with the given channel/output gpio_num/interrupt/source timer/frequency(Hz)/LEDC duty.

Parameters: `ledc_conf` -- Pointer of LEDC channel configure struct

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

`uint32_t ledc_find_suitable_duty_resolution(uint32_t src_clk_freq, uint32_t timer_freq)`

Helper function to find the maximum possible duty resolution in bits for `ledc_timer_config()`

Parameters:

- `src_clk_freq` -- LEDC timer source clock frequency (Hz) (See doxygen comments of `ledc_clk_cfg_t` or get from `esp_clk_tree_src_get_freq_hz()`)
- `timer_freq` -- Desired LEDC timer frequency (Hz)

Returns:

- 0 The timer frequency cannot be achieved
- Others The largest duty resolution value to be set

`esp_err_t ledc_timer_config(const ledc_timer_config_t *timer_conf)`

LEDC timer configuration Configure LEDC timer with the given source timer/frequency(Hz)/duty_resolution.

Parameters: `timer_conf` -- Pointer of LEDC timer configure struct

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Can not find a proper pre-divider number base on the given frequency and the current duty_resolution.
- ESP_ERR_INVALID_STATE Timer cannot be de-configured because timer is not configured or is not paused

`esp_err_t ledc_update_duty(ledc_mode_t speed_mode, ledc_channel_t channel)`

LEDC update channel parameters.

❗ Note

Call this function to activate the LEDC updated parameters. After `ledc_set_duty`, we need to call this function to update the settings. And the new LEDC parameters don't take effect until the next PWM cycle.

❗ Note

`ledc_set_duty`, `ledc_set_duty_with_hpoint` and `ledc_update_duty` are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is `ledc_set_duty_and_update`

❗ Note

If `CONFIG_LEDC_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Cache is disabled.

❗ Note

This function is allowed to run within ISR context.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

`esp_err_t ledc_set_pin(int gpio_num, ledc_mode_t speed_mode, ledc_channel_t channel)`

Set LEDC output gpio.

❗ Note

This function only routes the LEDC signal to GPIO through matrix, other LEDC resources initialization are not involved. Please use `ledc_channel_config()` instead to fully configure a LEDC channel.

Parameters:

- **gpio_num** -- The LEDC output gpio
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t ledc_stop(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t idle_level)

LEDC stop. Disable LEDC output, and set idle level.

❗ Note

If `CONFIG_LEDC_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Cache is disabled.

❗ Note

This function is allowed to run within ISR context.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
- **idle_level** -- Set output idle level after LEDC stops.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t ledc_set_freq(ledc_mode_t speed_mode, ledc_timer_t timer_num, uint32_t freq_hz)

LEDC set channel frequency (Hz)

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **timer_num** -- LEDC timer index (0-3), select from `ledc_timer_t`
- **freq_hz** -- Set the LEDC frequency

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Can not find a proper pre-divider number base on the given frequency and the current duty_resolution.

uint32_t ledc_get_freq(ledc_mode_t speed_mode, ledc_timer_t timer_num)

LEDC get channel frequency (Hz)

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **timer_num** -- LEDC timer index (0-3), select from `ledc_timer_t`

Returns:

- 0 error
- Others Current LEDC frequency

```
esp_err_t ledc_set_duty_with_hpoint(ledc_mode_t speed_mode, ledc_channel_t channel,
uint32_t duty, uint32_t hpoint)
```

LEDC set duty and hpoint value Only after calling ledc_update_duty will the duty update.

❗ Note

ledc_set_duty, ledc_set_duty_with_hpoint and ledc_update_duty are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_duty_and_update

❗ Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **duty** -- Set the LEDC duty, the range of duty setting is [0, (2**duty_resolution)]
- **hpoint** -- Set the LEDC hpoint value, the range is [0, (2**duty_resolution)-1]

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

```
int ledc_get_hpoint(ledc_mode_t speed_mode, ledc_channel_t channel)
```

LEDC get hpoint value, the counter value when the output is set high level.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t

Returns:

- LEDC_ERR_VAL if parameter error
- Others Current hpoint value of LEDC channel

```
esp_err_t ledc_set_duty(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t duty)
```

LEDC set duty This function do not change the hpoint value of this channel. if needed, please call `ledc_set_duty_with_hpoint`. only after calling `ledc_update_duty` will the duty update.

❗ Note

`ledc_set_duty`, `ledc_set_duty_with_hpoint` and `ledc_update_duty` are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is `ledc_set_duty_and_update`.

❗ Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
- **duty** -- Set the LEDC duty, the range of duty setting is [0, (2**duty_resolution)]

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

`uint32_t ledc_get_duty(ledc_mode_t speed_mode, ledc_channel_t channel)`

LEDC get duty This function returns the duty at the present PWM cycle. You shouldn't expect the function to return the new duty in the same cycle of calling `ledc_update_duty`, because duty update doesn't take effect until the next cycle.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`

Returns:

- LEDC_ERR_DUTY if parameter error
- Others Current LEDC duty

`esp_err_t ledc_set_fade(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t duty, ledc_duty_direction_t fade_direction, uint32_t step_num, uint32_t duty_cycle_num, uint32_t duty_scale)`

LEDC set gradient Set LEDC gradient, After the function calls the `ledc_update_duty` function, the function can take effect.

❗ Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
 - **duty** -- Set the start of the gradient duty, the range of duty setting is `[0, (2**duty_resolution)]`
 - **fade_direction** -- Set the direction of the gradient
 - **step_num** -- Set the number of the gradient
 - **duty_cycle_num** -- Set how many LEDC tick each time the gradient lasts
 - **duty_scale** -- Set gradient change amplitude
- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error

```
esp_err_t ledc_isr_register(void (*fn)(void*), void *arg, int intr_alloc_flags, ledc_isr_handle_t *handle)
```

Register LEDC interrupt handler, the handler is an ISR. The handler will be attached to the same CPU core that this function is running on.

- Parameters:**
- **fn** -- Interrupt handler function.
 - **arg** -- User-supplied argument passed to the handler function.
 - **intr_alloc_flags** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See `esp_intr_alloc.h` for more info.
 - **handle** -- Pointer to return handle. If non-NULL, a handle for the interrupt will be returned here.
- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error
 - ESP_ERR_NOT_FOUND Failed to find available interrupt source

```
esp_err_t ledc_timer_rst(ledc_mode_t speed_mode, ledc_timer_t timer_sel)
```

Reset LEDC timer.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **timer_sel** -- LEDC timer index (0-3), select from `ledc_timer_t`

Returns:

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

`esp_err_t ledc_timer_pause(ledc_mode_t speed_mode, ledc_timer_t timer_sel)`

Pause LEDC timer counter.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **timer_sel** -- LEDC timer index (0-3), select from `ledc_timer_t`

Returns:

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

`esp_err_t ledc_timer_resume(ledc_mode_t speed_mode, ledc_timer_t timer_sel)`

Resume LEDC timer.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **timer_sel** -- LEDC timer index (0-3), select from `ledc_timer_t`

Returns:

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

`esp_err_t ledc_bind_channel_timer(ledc_mode_t speed_mode, ledc_channel_t channel, ledc_timer_t timer_sel)`

Bind LEDC channel with the selected timer.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
 - **timer_sel** -- LEDC timer index (0-3), select from `ledc_timer_t`

Returns:

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

`esp_err_t ledc_set_fade_with_step(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t target_duty, uint32_t scale, uint32_t cycle_num)`

Set LEDC fade function.

Note

Call `ledc_fade_func_install()` once before calling this function. Call `ledc_fade_start()` after this to start fading.

Note

ledc_set_fade_with_step, ledc_set_fade_with_time and ledc_fade_start are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_fade_step_and_start

Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
 - **target_duty** -- Target duty of fading [0, (2**duty_resolution)]
 - **scale** -- Controls the increase or decrease step scale.
 - **cycle_num** -- increase or decrease the duty every cycle_num cycles

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

```
esp_err_t ledc_set_fade_with_time(ledc_mode_t speed_mode, ledc_channel_t channel,
uint32_t target_duty, int desired_fade_time_ms)
```

Set LEDC fade function, with a limited time.

Note

Call ledc_fade_func_install() once before calling this function. Call ledc_fade_start() after this to start fading.

Note

ledc_set_fade_with_step, ledc_set_fade_with_time and ledc_fade_start are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_fade_step_and_start

Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the

fade operation has finished.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
- **target_duty** -- Target duty of fading [0, (2**duty_resolution)]
- **desired_fade_time_ms** -- The intended time of the fading (ms).
Note that the actual time it takes to complete the fade could vary by a factor of up to 2x shorter or longer than the expected time due to internal rounding errors in calculations. Specifically:
 - The total number of cycles ($\text{total_cycle_num} = \text{desired_fade_time_ms} * \text{freq} / 1000$)
 - The difference in duty cycle ($\text{duty_delta} = |\text{target_duty} - \text{current_duty}|$) The fade may complete faster than expected if `total_cycle_num` larger than `duty_delta`. Conversely, it may take longer than expected if `total_cycle_num` is less than `duty_delta`. The closer the ratio of `total_cycle_num/duty_delta` (or its inverse) is to a whole number (the floor value), the more accurately the actual fade duration will match the intended time. If an exact fade time is expected, please consider to split the entire fade into several smaller linear fades. The split should make each fade step has a divisible `total_cycle_num/duty_delta` (or its inverse) ratio.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

`esp_err_t ledc_fade_func_install(int intr_alloc_flags)`

Install LEDC fade function. This function will occupy interrupt of LEDC module.

Parameters:

intr_alloc_flags -- Flags used to allocate the interrupt. One or multiple (ORred) `ESP_INTR_FLAG_*` values. See `esp_intr_alloc.h` for more info.

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Intr flag error
- ESP_ERR_NOT_FOUND Failed to find available interrupt source
- ESP_ERR_INVALID_STATE Fade function already installed

`void ledc_fade_func_uninstall(void)`

Uninstall LEDC fade function.

```
esp_err_t ledc_fade_start(ledc_mode_t speed_mode, ledc_channel_t channel, ledc_fade_mode_t fade_mode)
```

Start LEDC fading.

❗ Note

Call `ledc_fade_func_install()` once before calling this function. Call this API right after `ledc_set_fade_with_time` or `ledc_set_fade_with_step` before to start fading.

❗ Note

Starting fade operation with this API is not thread-safe, use with care.

❗ Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel number
- **fade_mode** -- Whether to block until fading done. See `ledc_types.h` `ledc_fade_mode_t` for more info. Note that this function will not return until fading to the target duty if `LEDC_FADE_WAIT_DONE` mode is selected.

Returns:

- `ESP_OK` Success
- `ESP_ERR_INVALID_STATE` Channel not initialized or fade function not installed.
- `ESP_ERR_INVALID_ARG` Parameter error.

```
esp_err_t ledc_set_duty_and_update(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t duty, uint32_t hpoint)
```

A thread-safe API to set duty for LEDC channel and return when duty updated.

❗ Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
 - **duty** -- Set the LEDC duty, the range of duty setting is [0, (2**duty_resolution)]
 - **hpoint** -- Set the LEDC hpoint value, the range is [0, (2**duty_resolution)-1]

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_STATE Channel not initialized
 - ESP_ERR_INVALID_ARG Parameter error
 - ESP_FAIL Fade function init error

```
esp_err_t ledc_set_fade_time_and_start(ledc_mode_t speed_mode, ledc_channel_t channel,
uint32_t target_duty, uint32_t desired_fade_time_ms, ledc_fade_mode_t fade_mode)
```

A thread-safe API to set and start LEDC fade function, with a limited time.

❗ Note

Call `ledc_fade_func_install()` once, before calling this function.

❗ Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters:

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
- **target_duty** -- Target duty of fading [0, (2**duty_resolution)]
- **desired_fade_time_ms** -- The intended time of the fading (ms).
Note that the actual time it takes to complete the fade could vary by a factor of up to 2x shorter or longer than the expected time due to internal rounding errors in calculations. Specifically:
 - The total number of cycles ($\text{total_cycle_num} = \text{desired_fade_time_ms} * \text{freq} / 1000$)
 - The difference in duty cycle ($\text{duty_delta} = |\text{target_duty} - \text{current_duty}|$) The fade may complete faster than expected if `total_cycle_num` larger than `duty_delta`. Conversely, it may take longer than expected if `total_cycle_num` is less than `duty_delta`. The closer the ratio of `total_cycle_num/duty_delta` (or its inverse) is to a whole number (the floor value), the more accurately the actual fade duration will match the intended time. If an exact fade time is expected, please consider to split the entire fade into several smaller linear fades. The split should make each fade step has a divisible `total_cycle_num/duty_delta` (or its inverse) ratio.

- **fade_mode** -- choose blocking or non-blocking mode

Returns:

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

`esp_err_t ledc_set_fade_step_and_start(ledc_mode_t speed_mode, ledc_channel_t channel, uint32_t target_duty, uint32_t scale, uint32_t cycle_num, ledc_fade_mode_t fade_mode)`

A thread-safe API to set and start LEDC fade function.

❗ Note

Call `ledc_fade_func_install()` once before calling this function.

❗ Note

For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
 - **target_duty** -- Target duty of fading [0, (2**duty_resolution)]
 - **scale** -- Controls the increase or decrease step scale.
 - **cycle_num** -- increase or decrease the duty every cycle_num cycles
 - **fade_mode** -- choose blocking or non-blocking mode

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error
 - ESP_ERR_INVALID_STATE Channel not initialized
 - ESP_FAIL Fade function init error

```
esp_err_t ledc_cb_register(ledc_mode_t speed_mode, ledc_channel_t channel, ledc_cbs_t *cbs, void *user_arg)
```

LEDC callback registration function.

❗ Note

The callback is called from an ISR, it must never attempt to block, and any FreeRTOS API called must be ISR capable.

- Parameters:**
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
 - **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from `ledc_channel_t`
 - **cbs** -- Group of LEDC callback functions
 - **user_arg** -- user registered data for the callback function

- Returns:**
- ESP_OK Success
 - ESP_ERR_INVALID_ARG Parameter error
 - ESP_ERR_INVALID_STATE Channel not initialized
 - ESP_FAIL Fade function init error

Structures

```
struct ledc_channel_config_t
```

Configuration parameters of LEDC channel for `ledc_channel_config` function.

Public Members

```
int gpio_num
```

the LEDC output gpio_num, if you want to use gpio16, gpio_num = 16

ledc_mode_t speed_mode

LEDC speed speed_mode, high-speed mode (only exists on esp32) or low-speed mode

ledc_channel_t channel

LEDC channel (0 - LEDC_CHANNEL_MAX-1)

ledc_intr_type_t intr_type

Deprecated:

, no need to explicitly configure interrupt, handled in the driver

ledc_timer_t timer_sel

Select the timer source of channel (0 - LEDC_TIMER_MAX-1)

uint32_t duty

LEDC channel duty, the range of duty setting is [0, (2**duty_resolution)]

int hpoint

LEDC channel hpoint value, the range is [0, (2**duty_resolution)-1]

ledc_sleep_mode_t sleep_mode

choose the desired behavior for the LEDC channel in Light-sleep

struct ledc_channel_config_t::ledc_channel_flags flags

Extra configuration flags for LEDC channel

bool deconfigure

Set this field to de-configure a LEDC channel which has been configured before The driver only does limited action to release the pins occupied by this channel only. When this field is set, gpio_num, timer_sel, duty, hpoint, sleep_mode, flags fields are ignored.

struct ledc_channel_flags

Extra configuration flags for LEDC channel.

Public Members**unsigned int output_invert**

Enable (1) or disable (0) gpio output invert

struct ledc_timer_config_t

Configuration parameters of LEDC timer for `ledc_timer_config` function.

Public Members

`ledc_mode_t` `speed_mode`

LEDC speed `speed_mode`, high-speed mode (only exists on esp32) or low-speed mode

`ledc_timer_bit_t` `duty_resolution`

LEDC channel duty resolution

`ledc_timer_t` `timer_num`

The timer source of channel (0 - LEDC_TIMER_MAX-1)

`uint32_t` `freq_hz`

LEDC timer frequency (Hz)

`ledc_clk_cfg_t` `clk_cfg`

Configure LEDC source clock from `ledc_clk_cfg_t`. Note that LEDC_USE_RC_FAST_CLK and LEDC_USE_XTAL_CLK are non-timer-specific clock sources. You can not have one LEDC timer uses RC_FAST_CLK as the clock source and have another LEDC timer uses XTAL_CLK as its clock source. All chips except esp32 and esp32s2 do not have timer-specific clock sources, which means clock source for all timers must be the same one.

`bool` `deconfigure`

Set this field to de-configure a LEDC timer which has been configured before Note that it will not check whether the timer wants to be de-configured is binded to any channel. Also, the timer has to be paused first before it can be de-configured. When this field is set, `duty_resolution`, `freq_hz`, `clk_cfg` fields are ignored.

`struct` `ledc_cb_param_t`

LEDC callback parameter.

Public Members

`ledc_cb_event_t` `event`

Event name

`uint32_t` `speed_mode`

Speed mode of the LEDC channel group

`uint32_t` `channel`

uint32_t duty

LEDC current duty of the channel, the range of duty is [0, (2**duty_resolution)]

struct ledc_cbs_t

Group of supported LEDC callbacks.

📌

Note

The callbacks are all running under ISR environment

Public Members

ledc_cb_t fade_cb

LEDC fade_end callback function

Macros

LEDC_ERR_DUTY

LEDC_ERR_VAL

Type Definitions

typedef intr_handle_t ledc_isr_handle_t

typedef bool (*ledc_cb_t)(const ledc_cb_param_t *param, void *user_arg)

Type of LEDC event callback.

- Param param:

LEDC callback parameter
- Param user_arg:

User registered data
- Return:

Whether a high priority task has been waken up by this function

Enumerations

enum ledc_sleep_mode_t

Strategies to be applied to the LEDC channel during system Light-sleep period.

Values:

enumerator LEDC_SLEEP_MODE_NO_ALIVE_NO_PD

The default mode: no LEDC output, and no power off the LEDC power domain.

enumerator LEDC_SLEEP_MODE_NO_ALIVE_ALLOW_PD

The low-power-consumption mode: no LEDC output, and allow to power off the LEDC power domain. This can save power, but at the expense of more RAM being consumed to save register context. This option is only available on targets that support TOP domain to be powered down.

enumerator LEDC_SLEEP_MODE_KEEP_ALIVE

The high-power-consumption mode: keep LEDC output when the system enters Light-sleep.

enumerator LEDC_SLEEP_MODE_INVALID

Invalid LEDC sleep mode strategy

enum ledc_cb_event_t

LEDC callback event type.

Values:

enumerator LEDC_FADE_END_EVT

LEDC fade end event

Header File

- [components/esp_hal_ledc/include/hal/ledc_types.h](#)
- This header file can be included with:

```
#include "hal/ledc_types.h"
```

- This header file is a part of the API provided by the `esp_hal_ledc` component. To declare that your component depends on `esp_hal_ledc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_hal_ledc
```

or

```
PRIV_REQUIRES esp_hal_ledc
```

Type Definitions

`typedef soc_periph_ledc_clk_src_legacy_t ledc_clk_cfg_t`

LEDC clock source configuration struct.

In theory, the following enumeration shall be placed in LEDC driver's header. However, as the next enumeration, `ledc_clk_src_t`, makes the use of some of these values and to avoid mutual inclusion of the headers, we must define it here.

Enumerations

`enum ledc_mode_t`

Values:

`enumerator LEDC_HIGH_SPEED_MODE`

LEDC high speed speed_mode

`enumerator LEDC_LOW_SPEED_MODE`

LEDC low speed speed_mode

`enumerator LEDC_SPEED_MODE_MAX`

LEDC speed limit

`enum ledc_intr_type_t`

Values:

`enumerator LEDC_INTR_DISABLE`

Disable LEDC interrupt

`enumerator LEDC_INTR_FADE_END`

Enable LEDC interrupt

`enumerator LEDC_INTR_MAX`

`enum ledc_duty_direction_t`

Values:

`enumerator LEDC_DUTY_DIR_DECREASE`

LEDC duty decrease direction

`enumerator LEDC_DUTY_DIR_INCREASE`

LEDC duty increase direction

enumerator LEDC_DUTY_DIR_MAX

enum ledc_slow_clk_sel_t

LEDC global clock sources.

Values:

enumerator LEDC_SLOW_CLK_RC_FAST

LEDC low speed timer clock source is RC_FAST clock

enumerator LEDC_SLOW_CLK_APB

LEDC low speed timer clock source is 80MHz APB clock

enum ledc_clk_src_t

LEDC timer-specific clock sources.

Note: Setting numeric values to match ledc_clk_cfg_t values are a hack to avoid collision with LEDC_AUTO_CLK in the driver, as these enums have very similar names and user may pass one of these by mistake.

Values:

enumerator LEDC_REF_TICK

LEDC timer clock divided from reference tick (1Mhz)

enumerator LEDC_APB_CLK

LEDC timer clock divided from APB clock (80Mhz)

enumerator LEDC_SCLK

Selecting this value for LEDC_TICK_SEL_TIMER let the hardware take its source clock from LEDC_APB_CLK_SEL

enum ledc_timer_t

Values:

enumerator LEDC_TIMER_0

LEDC timer 0

enumerator LEDC_TIMER_1

LEDC timer 1

enumerator LEDC_TIMER_2

LEDC timer 2

enumerator LEDC_TIMER_3

LEDC timer 3

enumerator LEDC_TIMER_MAX

enum ledc_channel_t

Values:

enumerator LEDC_CHANNEL_0

LEDC channel 0

enumerator LEDC_CHANNEL_1

LEDC channel 1

enumerator LEDC_CHANNEL_2

LEDC channel 2

enumerator LEDC_CHANNEL_3

LEDC channel 3

enumerator LEDC_CHANNEL_4

LEDC channel 4

enumerator LEDC_CHANNEL_5

LEDC channel 5

enumerator LEDC_CHANNEL_6

LEDC channel 6

enumerator LEDC_CHANNEL_7

LEDC channel 7

enumerator LEDC_CHANNEL_MAX

enum ledc_timer_bit_t

Values:

***enumerator* LEDC_TIMER_1_BIT**

LEDC PWM duty resolution of 1 bits

***enumerator* LEDC_TIMER_2_BIT**

LEDC PWM duty resolution of 2 bits

***enumerator* LEDC_TIMER_3_BIT**

LEDC PWM duty resolution of 3 bits

***enumerator* LEDC_TIMER_4_BIT**

LEDC PWM duty resolution of 4 bits

***enumerator* LEDC_TIMER_5_BIT**

LEDC PWM duty resolution of 5 bits

***enumerator* LEDC_TIMER_6_BIT**

LEDC PWM duty resolution of 6 bits

***enumerator* LEDC_TIMER_7_BIT**

LEDC PWM duty resolution of 7 bits

***enumerator* LEDC_TIMER_8_BIT**

LEDC PWM duty resolution of 8 bits

***enumerator* LEDC_TIMER_9_BIT**

LEDC PWM duty resolution of 9 bits

***enumerator* LEDC_TIMER_10_BIT**

LEDC PWM duty resolution of 10 bits

***enumerator* LEDC_TIMER_11_BIT**

LEDC PWM duty resolution of 11 bits

***enumerator* LEDC_TIMER_12_BIT**

LEDC PWM duty resolution of 12 bits

***enumerator* LEDC_TIMER_13_BIT**

LEDC PWM duty resolution of 13 bits

***enumerator* LEDC_TIMER_14_BIT**

LEDC PWM duty resolution of 14 bits

***enumerator* LEDC_TIMER_15_BIT**

LEDC PWM duty resolution of 15 bits

***enumerator* LEDC_TIMER_16_BIT**

LEDC PWM duty resolution of 16 bits

***enumerator* LEDC_TIMER_17_BIT**

LEDC PWM duty resolution of 17 bits

***enumerator* LEDC_TIMER_18_BIT**

LEDC PWM duty resolution of 18 bits

***enumerator* LEDC_TIMER_19_BIT**

LEDC PWM duty resolution of 19 bits

***enumerator* LEDC_TIMER_20_BIT**

LEDC PWM duty resolution of 20 bits

***enumerator* LEDC_TIMER_BIT_MAX**

***enum* ledc_fade_mode_t**

Values:

***enumerator* LEDC_FADE_NO_WAIT**

LEDC fade function will return immediately

***enumerator* LEDC_FADE_WAIT_DONE**

LEDC fade function will block until fading to the target duty

***enumerator* LEDC_FADE_MAX**

Was this page helpful?



